

# Guia 1



**PARTICIPANTES:** Jamile Victoria Ferreira da Silva

Noemia Soares Alves dos Santos

Lara Gabrielly dos Santos

Camila Delazari Silva

Enzo Pietro Inácio Rodrigues

Pablo Guilherme Siqueira da Silva

Yuri Valadares

João Vitor Alves Nunes

Rennan Roberto Caldeira Vieira

Arthur Vinícius Nunes Matias de Araújo

Matheus Oliveira de Jesus

## **DOCUMENTAÇÃO DE APRESENTAÇÃO**

**TEMA:** App para marcar consultas em uma clínica medica

### **CONTEÚDO TEÓRICO:**

**1- OBJETIVO DO SISTEMA:** Automatizar o agendamento de consultas, eliminando conflitos de horarios, reduzindo faltas e organizando fluxo de pacientes, medicos e recursos da clinica

O app life med foi criado com a intenção de ajudar os pacientes de uma forma mais prática para agendar consultas ao invés de ir na clínica para fazer o agendamento.

### **FUNÇÃO DO APP**

Ele te ajuda a fazer um agendamento de uma consulta mais rápida, de forma mais fácil, isso evita da sua ida até a clínica para realizar as suas consultas, isso também ajuda na experiência dos clientes.

### **PRINCIPAL FUNCIONALIDADE**

\* atendimento online:

## **IMPORTÂNCIA DA TECNOLOGIA NA SAÚDE**

Hoje em dia a tecnologia é muito avançada para qualquer tipo de coisa que você precisa, por isso a tecnologia na saúde é muito importante porque ajuda na facilidade de uma comunicação mais rápida e melhora na parte da segurança dos seus dados.

## **SEGURANÇA DE DADOS**

O app tem a necessidade de proteger os dados dos usuários, de forma segura e com privacidade das informações médicas, isso para a confiança dos pacientes na hora de acessar o app.

## **EXEMPLOS**

- \* colocar os seus dados para entrar ou criar sua conta para poder marcar sua consulta.
- \* o cliente pode marcar sua consulta sem precisar ir a clínica e sem precisar ligar lá.
- \* marca a consulta pelo celular ou notebook.
- \* escolher o dia e horário da sua consulta.
- \* ver consultas já agendadas.
- \* confirmar sua consulta pelo app.
- \* marcar consulta com o clínico geral ou outra especialidade.

**VARIÁVEIS E VETORES:** As variáveis, podem armazenar informações como: nomes, horários, médicos, consultas e locais e os vetores seriam responsáveis por armazenar o conjuntos dessas informações criando uma consulta específica com os dados que cada paciente e responsável por fazer a consulta devem para que o agendamento seja feito da melhor forma

**OS RISCOS DO AGENDAMENTAMENTO MANUAL:** Existem várias diferenças entre esses modelos, mas a principal está no tipo de controle. A agenda física e a planilha de agendamento dependem de controle manual, o que abre espaço para: rasuras, e versões desatualizadas.

**CÓDIGOS:**

```
let usuariosCadastrados = JSON.parse(localStorage.getItem("usuariosCadastrados"))
|| [
  { email: "user@lifemed.com", senha: "123", nome: "LifeMed user" }
];
```

```
function toggleAuthMode(isRegistering) {
  const signInForm = document.getElementById("signInForm");
  const signUpForm = document.getElementById("signUpForm");
```

```
  document.getElementById("authLoginError").innerText = "";
  document.getElementById("authRegisterError").innerText = "";
```

```
  if (isRegistering) {
    signInForm.style.display = "none";
    signUpForm.style.display = "block";
  } else {
    signInForm.style.display = "block";
    signUpForm.style.display = "none";
  }
}
```

```
function fazerLogin() {
  const emailInput = document.getElementById("loginEmail").value.trim();
  const senhaInput = document.getElementById("loginSenha").value;
  const errorElement = document.getElementById("authLoginError");
```

```
  errorElement.innerText = "";
```

```
  if (!emailInput || !senhaInput) {
    errorElement.innerText = "Por favor, preencha todos os campos.";
    return;
  }
```

```
  const usuarioValido = usuariosCadastrados.find(
    u => u.email.toLowerCase() === emailInput.toLowerCase() && u.senha ===
    senhaInput
  );
```

```
  if (usuarioValido) {
    // Injeta os dados do usuário logado diretamente nos campos do seu HTML
    document.querySelector(".user-name").innerText = usuarioValido.nome;
    document.getElementById("patientName").value = usuarioValido.nome;
    document.getElementById("patientEmail").value = usuarioValido.email;
```

```
    // Esconde a tela de login e mostra o sistema
    document.getElementById("authScreen").style.display = "none";
```

```

    document.getElementById("mainSystem").style.display = "block";
    document.getElementById("sidebar").style.display = "flex";
  } else {
    errorElement.innerText = "E-mail ou senha incorretos.";
  }
}

```

```

function fazerCadastro() {
  const nome = document.getElementById("registerName").value.trim();
  const email = document.getElementById("registerEmail").value.trim();
  const senha = document.getElementById("registerSenha").value;
  const errorElement = document.getElementById("authRegisterError");

  errorElement.innerText = "";

  if (!nome || !email || !senha) {
    errorElement.innerText = "Por favor, preencha todos os campos.";
    return;
  }

  const emailExiste = usuariosCadastrados.some(u => u.email.toLowerCase() ===
email.toLowerCase());
  if (emailExiste) {
    errorElement.innerText = "Este e-mail já está cadastrado.";
    return;
  }

  // Adiciona no array temporário
  usuariosCadastrados.push({ nome, email, senha });

  // ATUALIZA O BANCO DE DADOS (Grava permanentemente no navegador)
  localStorage.setItem("usuariosCadastrados", JSON.stringify(usuariosCadastrados));

  // Volta para a tela de login com mensagem de sucesso
  toggleAuthMode(false);
  document.getElementById("authLoginError").style.color = "var(--teal-400)";
  document.getElementById("authLoginError").innerText = "Conta criada! Faça seu
login.";

  setTimeout(() => {
    document.getElementById("authLoginError").style.color = "";
  }, 4000);
}

let etapaAtual = 1;
function goStep(step) {
  // 1. Limpa as mensagens de erro antigas

```

```
document.getElementById("nameError").innerText = "";
document.getElementById("emailError").innerText = "";
document.getElementById("phoneError").innerText = "";
document.getElementById("cpfError").innerText = "";
document.getElementById("dobError").innerText = "";
```

// 2. Validações de campos

```
if (step === 2) {
  const nome = document.getElementById("patientName").value;
  const email = document.getElementById("patientEmail").value;
  const phone = document.getElementById("patientPhone").value;
  const cpf = document.getElementById("patientCPF").value;
  const dob = document.getElementById("patientDOB").value;
  let possuiErro = false;
  if (!nome) { document.getElementById("nameError").innerText = "Preencha o nome"; possuiErro = true; }
  if (!email) { document.getElementById("emailError").innerText = "Preencha o e-mail"; possuiErro = true; }
  if (!phone) { document.getElementById("phoneError").innerText = "Preencha o telefone"; possuiErro = true; }
  if (!cpf) { document.getElementById("cpfError").innerText = "Preencha o CPF"; possuiErro = true; }
  if (!dob) { document.getElementById("dobError").innerText = "Preencha a data de nascimento"; possuiErro = true; }
  if (possuiErro) return;
}
if (step === 3) {
  const doctor = document.getElementById("doctor").value;
  let possuiErro = false;
```

```
if (!doctor) { document.getElementById("doctorError").innerText = "Selecione um médico"; possuiErro = true; }
if (possuiErro) return;
}
if (step === 4) {
  const consultTime = document.getElementById("consultTime").value;
  const consultDate = document.getElementById("consultDate").value;
  const notes = document.getElementById("notes").value;
  let possuiErro = false;
  if (!consultDate) { document.getElementById("dateError").innerText = "Selecione a data da consulta"; possuiErro = true; }
  if (!consultTime) { document.getElementById("timeError").innerText = "Selecione o horário"; possuiErro = true; }
  if (!notes) { document.getElementById("notesError").innerText = "Campo Obrigatório"; possuiErro = true; }
```

```
    if (possuiErro) return;
}
```

```
etapaAtual = step;
```

```
// 3. Controla os passos do formulário
```

```
document.getElementById("step1").style.display = "none";
document.getElementById("step2").style.display = "none";
document.getElementById("step3").style.display = "none";
document.getElementById("step4").style.display = "none";
```

```
// GARANTE que a mensagem de sucesso comece sumida enquanto navega nos passos
```

```
document.getElementById("successMsg").style.display = "none";
```

```
document.getElementById(step${step}).style.display = "block";
```

```
// 4. Atualiza as bolinhas do topo
```

```
const todosPassos = document.querySelectorAll(".steps .step");
todosPassos.forEach((elementoPasso) => {
    const numeroPasso = parseInt(elementoPasso.getAttribute("data-step"));
    elementoPasso.classList.remove("active", "done");
    if (numeroPasso === step) {
        elementoPasso.classList.add("active");
    } else if (numeroPasso < step) {
        elementoPasso.classList.add("done");
    }
});
```

```
// 5. Atualiza a revisão do Passo 4
```

```
const patientName = document.getElementById("patientName").value;
const patientPhone = document.getElementById("patientPhone").value;
const patientEmail = document.getElementById("patientEmail").value;
const doctor = document.getElementById("doctor").value;
const consultDate = document.getElementById("consultDate").value;
const consultTime = document.getElementById("consultTime").value;
const consultTypeInput =
document.querySelector('input[name="consultType"]:checked');
const consultType = consultTypeInput ? consultTypeInput.value : "Presencial";
const specialtySelecionada =
document.querySelector('input[name="specialty"]:checked');
let specialty = "Não selecionada";
if (specialtySelecionada) {
    specialty = specialtySelecionada.parentElement.innerText.trim();
}

document.getElementById("rev-name").innerText = patientName;
```

```
document.getElementById("rev-phone").innerText = patientPhone;
document.getElementById("rev-email").innerText = patientEmail;
document.getElementById("rev-specialty").innerText = specialty;
document.getElementById("rev-doctor").innerText = doctor;
document.getElementById("rev-type").innerText = consultType;
document.getElementById("rev-date").innerText = consultDate;
document.getElementById("rev-time").innerText = consultTime;
}
```

```
function finalizarAgendamento() {
  // Esconde o formulário completamente
  document.getElementById("appointmentForm").style.display = "none";

  // MOSTRA a mensagem restaurando o layout padrão do CSS (centralizado)
  document.getElementById("successMsg").style.display = "";
}
```

```
function resetForm() {
  document.getElementById("appointmentForm").reset();

  // Esconde a tela de sucesso
  document.getElementById("successMsg").style.display = "none";

  // Mostra o formulário novamente
  document.getElementById("appointmentForm").style.display = "block";

  const usuarioLogadoNome = document.querySelector(".user-name").innerText;
  const usuarioLogadoEmail = document.getElementById("patientEmail").value;
  if(usuarioLogadoNome !== "Paciente") {
    document.getElementById("patientName").value = usuarioLogadoNome;
    document.getElementById("patientEmail").value = usuarioLogadoEmail;
  }

  goStep(1);
}
```

```
function mascaraTelefone(input) {
  let valor = input.value.replace(/\D/g, "");
  if (valor.length > 11) valor = valor.slice(0, 11);

  if (valor.length > 7) {
    input.value = (${valor.slice(0, 2)}) ${valor.slice(2, 7)}-${valor.slice(7)};
  } else if (valor.length > 2) {
    input.value = (${valor.slice(0, 2)}) ${valor.slice(2)};
  } else if (valor.length > 0) {
    input.value = (${valor});
  } else {
    input.value = "";
  }
}
```



```
}  
}
```

```
function mascaraCPF(input) {
```

```
function mascaraCPF(input) {  
  let valor = input.value.replace(/\D/g, "");  
  if (valor.length > 11) valor = valor.slice(0, 11);  
  
  if (valor.length > 9) {  
    input.value = `${valor.slice(0, 3)}${valor.slice(3, 6)}${valor.slice(6, 9)}${valor.slice(9)};`  
  } else if (valor.length > 6) {  
    input.value = `${valor.slice(0, 3)}${valor.slice(3, 6)}${valor.slice(6)};`  
  } else if (valor.length > 3) {  
    input.value = `${valor.slice(0, 3)}${valor.slice(3)};`  
  } else if (valor.length > 0) {  
    input.value = `${valor};`  
  } else {  
    input.value = "";  
  }  
}
```

**CÓDIGO JAVASCRIPT** : Gerencia o fluxo de um sistema web de agendamento de consultas médicas. Ele lida com o cadastro e login de usuarios e controla um formulário de agendamento dividido em 4 etapas

## 1. Banco de Dados Local (LocalStorage)

Logo no início, o código cria um "banco de dados" fictício no navegador:

```
javascript  
let usuariosCadastrados = JSON.parse(localStorage.getItem("usuariosCadastrados")) || [ ...  
];
```

\* \*O que faz:\* Ele tenta buscar uma lista de usuários já cadastrados no localStorage. Se não encontrar nada (primeiro acesso), ele cria uma lista padrão contendo um usuário de teste: user@lifemed.com com a senha 123.

---

- \* \*O que faz:\* Alterna a interface da tela entre o formulário de Login e o de Cadastro.
- \* \*Passo a passo:\* Limpa as mensagens de erro anteriores e, se isRegistering for verdadeiro, esconde a tela de login e mostra a de cadastro. Se for falso, faz o inverso.

### fazerCadastro()

- \* \*O que faz:\* Cria uma nova conta no sistema.
- \* \*Passo a passo:

  1. Pega os valores digitados (Nome, E-mail e Senha) e remove espaços em branco inúteis (.trim()).
  2. Verifica se algum campo ficou vazio. Se sim, exibe um erro e para a execução.
  3. Verifica se o e-mail digitado já existe na lista (usuariosCadastrados). Se já existir, avisa o usuário e para.
  4. Se estiver tudo certo, adiciona o novo usuário no array.
  5. \*Salva permanentemente\* atualizando o localStorage com o novo usuário.
  6. Redireciona o usuário para a tela de login exibindo uma mensagem verde de sucesso que some após 4 segundos.

## 2. Sistema de Autenticação (Login e Cadastro)

- \* O que faz:\* Valida o acesso do usuário para liberar o sistema.
- \* \*Passo a passo:

  1. Captura o e-mail e a senha digitados.
  2. Procura no array de usuários se existe alguém com o mesmo e-mail (ignorando maiúsculas/minúsculas) e a mesma senha.
  3. \*Se o usuário for válido:\* Preenche automaticamente os campos de perfil do sistema com o nome e e-mail desse usuário, esconde a tela de autenticação (authScreen) e exibe o sistema principal (mainSystem e sidebar).
  4. \*Se for inválido:\* Exibe a mensagem "E-mail ou senha incorretos."

[16:01, 16/05/2026] Gabriel Curso: # 3. Fluxo do Formulário de Agendamento (Multi-step)

### goStep(step)

Esta é a função principal que controla o formulário de agendamento em 4 passos. Ela é disparada toda vez que o usuário clica em "Avançar" ou "Voltar".

- \* \*Passo a passo:

  1. \*Limpeza:\* Apaga todas as mensagens de erro de validação que existiam na tela.
  2. \*Validação do Passo 1 (Indo para o Passo 2):\* Verifica se o usuário preencheu Nome, E-mail, Telefone, CPF e Data de Nascimento. Se faltar algo, exibe o erro e impede o avanço.
  3. \*Validação do Passo 2 (Indo para o Passo 3):\* Verifica se um médico foi selecionado.
  4. \*Validação do Passo 3 (Indo para o Passo 4):\* Verifica se a data, o horário e as notas (motivo da consulta) foram preenchidos.
  5. \*Troca de Telas:\* Esconde todos os passos (step1 ao step4) e exibe apenas o passo atual correspondente ao número do argumento step.

6. \*Atualização Visual:\* Altera o estilo visual das "bolinhas" (indicadores de progresso) no topo da página, adicionando a classe active para o passo atual e done para os passos que já ficaram para trás.

7. \*Tela de Revisão (Passo 4):\* Se o usuário chegar ao passo 4, a função coleta \*todos\* os dados preenchidos nos passos anteriores (incluindo especialidade médica e tipo de consulta se é Presencial ou Teleconsulta) e os joga em elementos de texto na tela (rev-name, rev-phone, etc.) para que o paciente revise os dados antes de confirmar.

---

## ## 4. Finalização e Reinício

### ### finalizarAgendamento()

\* \*O que faz:\* Conclui o processo.

\* \*Passo a passo:\* Esconde...



# Guia 2



# Guia 3

